PROJECT AND TEAM INFORMATION

Project Title

A C++ Based Implementation of Fundamental Machine Learning Algorithms

Team Name: *Team#*	*Team Vita/Route DSCPP-III-2026-T170*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Kotiyal, Shubhangani - 2510370294 2510370294@geu.ac.in Section: ML2* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Agarwal, Ishanaya - 2510380249 2510380249@geu.ac.in Section: DS1* 

Team Name: *Team#*	*Team Vita/Route DSCPP-III-2026-T170*
Team member 3 *(Last Name, name: student ID: email, picture):*	*Joshi, Harshita - 2510380118 2510380118 @geu.ac.in* *Section: DS2* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

ML-Core is an educational project that implements fundamental machine learning algorithms from scratch so that their mathematical operations can be studied directly. The planned system covers regression, classification, clustering and dimensionality reduction through Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering and PCA. The implementation language has changed from C to C++, enabling classes and encapsulation while retaining explicit mathematical calculations.

The current prototype implements simple Linear Regression for one numerical input feature. It accepts training pairs, calculates the least-squares slope and intercept, predicts an output for a new input and reports training mean squared error. Standard C++ containers and mathematical functions support the implementation; no external machine learning library performs the fitting. Future work will add the remaining algorithms, reusable input utilities, an integrated interface and independent validation.

Updated Project Approach and Architecture

Each algorithm is implemented as a separate C++ class in its own code file. All six algorithms—Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA—have been coded. Each

program contains its own input handling, calculations, and output functions rather than using a shared utility layer. CSV dataset preparation remains pending, after which the programs will be tested using the prepared datasets.

Data Input: Read and validate numerical data. CSV file-reading code has been added to Linear Regression and KNN; the corresponding dataset files still need to be prepared.

Algorithm Classes: Separate classes implement the calculations required for regression, classification, clustering, and dimensionality reduction. Coding is complete for all six algorithms, while dataset-based testing and validation remain pending.

Evaluation and Output: The programs display results in the console. Linear Regression calculates training Mean Squared Error, while KNN displays the nearest neighbours and predicted class. Comparison with standard implementations such as Scikit-learn remains pending.

The project has moved from a C-based approach to separate C++ classes. Private data members store model parameters and data, while public member functions perform the required operations. The Linear Regression and KNN implementations use fixed-size arrays. Linear Regression uses a direct least-squares solution rather than gradient descent, so no learning rate or iteration count is required.

The implemented Linear Regression and KNN programs demonstrate classes, objects, constructors, encapsulation, member functions, arrays, and file handling. Array traversal and numerical accumulation support regression calculations, while distance calculations and bubble sorting support nearest-neighbour selection. The programs use file streams and string streams to read numerical CSV data. Dataset-based execution and validation will be completed once the CSV files are prepared.

Tasks Completed

Task Completed	**Team Member**
Linear Regression and KNN; Prototype implemented C++ class with fit, predict and MSE methods	Shubhangani Kotiyal
SVM and K-Means	Ishanaya Agarwal

Task Completed	Team Member
Hierarchical Clustering and PCA	Harshita Joshi
The first model class, interactive program, sample input, captured output, self-tests and usage guide have been prepared of every syllabus topic to a specific module	All members
The program compiled using C++17 with warning flags enabled and produced no compiler warnings. The checks were executed in the preparation environment. These artifacts are ready for team review and a local rerun; they are not evidence of prior individual commits.	All members

Testing and Challenges

Validation currently uses synthetic inputs with independently hand-calculable expectations. The included –test mode reports nine passing checks. Scikit-learn comparison and evaluation on held-out real data have not yet been performed. Test case: Exact line. Expected result: m = 2; b = 1; predict(6) = 13; MSE = 0. Observed: Pass. Test case: Negative slope. Expected result: m = −2; b = 5. Observed: Pass. Test case: Constant target. Expected result: m = 0; predict(4) = 7. Observed: Pass. Test case: Noisy data. Expected result: m = 1.5; b = 1/3; MSE = 1/18. Observed: Pass. Test case: Identical x values. Expected result: Reject fitting. Observed: Pass. Test case: Unequal input lengths. Expected result: Reject fitting. Observed: Pass. Test case: One training pair. Expected result: Reject fitting. Observed: Pass. Test case: Empty training data. Expected result: Reject fitting. Observed: Pass. Test case: Prediction before fitting. Expected result: Reject prediction. Observed: Pass.

Training input: (1, 3), (2, 5), (3, 7), (4, 9), (5, 11). Prediction input: x = 6. The captured run produced the following result: Slope: 2, Intercept: 1, Equation: y = (2) * x + (1), Training MSE: 0, Predicted y: 13.

The demonstration data lie exactly on y = 2x + 1, so zero training error is expected This verifies the formula for a controlled example; it does not establish predictive performance on real data. The noisy-data test checks a nonzero residual case. The prototype supports

one input feature, uses console input, and does not yet provide CSV loading, plots, multiple regression or a train/test split. A local team demonstration and screenshots remain pending. No success rate is claimed for the unimplemented algorithms, and no overall project completion percentage is assigned.

The immediate design task is converting the C-oriented proposal into a consistent C++ class structure while preserving the from-scratch mathematical focus. The prototype addresses invalid inputs and the zero denominator case, but broader numerical testing and real datasets are still needed. Shared utilities must be agreed before the remaining modules are integrated.

Tasks Pending

Day	Task Pending	Team Member
8 October	Begin or continue assigned modules	Each module owner
9 October	Integrate available code; run tests; capture outputs; update report and PPT	All members
10 October	Verify files and upload current Phase II deliverables	Each member from own account
11 to 16 October	Resolve defects and practise the demonstration and viva Phase II evaluation	All members

Project Outcome/Deliverables

The following are the key outcomes expected at the end of the project:

- A C++ based implementation of fundamental machine learning algorithms including Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering and PCA, implemented from scratch to study their mathematical operations directly.

- A modular object-oriented architecture in which each machine learning algorithm is implemented as a separate C++ class, with shared utilities planned for numerical data loading, matrix operations and distance calculations.

- A working Linear Regression prototype that accepts training data, calculates the slope and intercept using the least-squares method, predicts an output for a new input and calculates the training mean squared error.

- Input validation and error handling for cases such as empty training data, unequal input lengths, identical input values, insufficient training data and prediction before model fitting.

- Testing and validation of the implemented Linear Regression model, with nine self-tests successfully passed, covering exact, negative-slope, constant-target, noisy-data and invalid-input cases.

- An integrated console-based demonstration providing numerical input, model results and evaluation metrics, with shared CSV input and an integrated menu planned for the complete system.

- Complete documentation of the algorithms and implementation, including their mathematical calculations, object-oriented design, testing results and project architecture.

- Final tested algorithms, shared numerical and data utilities, an integrated demonstration, results and documentation as the final project deliverables.

Progress Overview

Codebase Information

Repository: https://github.com/Shubhangani-Kotiyal/PBL-ml-algorithms-in-c

Branches: The repository name currently refers to C. Its README and project title should state the new C++ implementation. Repository contents, branches and contribution counts have not been verified. No remote changes were made while preparing these files.

Layout: The current package contains linear_regression.cpp, sample_input.txt, sample_output.txt, test_results.txt and README.md. The proposed location is src/linear_regression/, with the report and presentation in docs/. Members will commit their own reviewed work from their own accounts

Deliverables Progress

- **Project architecture and requirements:** In progress. The Phase II design has been updated from the C-oriented proposal to a C++ class-based architecture, with each algorithm planned as a separate C++ class.
- **Linear Regression:** Completed. A standalone C++ Linear Regression prototype has been implemented with fitting, prediction and MSE calculation.
- **Input Validation & Testing** Completed: Sample count, finite values, vector checks and invalid fitting conditions have been handled. Nine out of nine self-tests have passed, including valid training cases and invalid-input cases.
- **KNN and SVM** In progress: Assigned in Phase I; implementation has not yet been completed.
- **K-Means, Hierarchical Clustering and PCA** In Progress: Assigned in Phase I; implementation has not yet been completed.
- **Shared numerical utilities** Pending: Reusable matrix and distance operations are planned but have not yet been implemented.
- **Shared CSV input and integrated menu** Pending: The architecture is planned, but CSV loading and the integrated interface have not yet been implemented.
- **Charts and exported results** Pending: Console results and training MSE are implemented; charts remain pending.
- **GitHub upload and team integration** Pending: Prepared files have not yet been pushed, and the remaining algorithms and cross-library validation are still pending.
- **Final project deliverables** Pending: The final scope includes six tested algorithms, shared numerical and data utilities, an integrated demonstration, results and documentation. Currently available are the standalone Linear Regression prototype, its checks and the Phase II progress report.